Not All Embeddings Are Gaussian: Revisiting Adaptive HNSW Search: [Experiments & Analysis]

Anonymous Author(s)

Abstract

Ada-ef (SIGMOD 2026) sets the search breadth (*ef*) of each HNSW query from a difficulty score. The score relies on an assumption: that a query's similarities to the database follow a Gaussian distribution. While the original paper checks this assumption on individual dimensions of two text datasets, we evaluate what the score actually depends on, the distribution of each query's similarities, across 41 public datasets. The Gaussian fits most text embeddings but departs significantly on nine datasets, six of which are learned embeddings from image, audio and recommendation models. Where it departs and queries share the data's modality (eight datasets), Ada-ef's score loses much of its ability to rank queries by difficulty; on SIFT and Yambda it assigns every query the same *ef*. Its released implementation then does more work than the best hindsight-tuned fixed *ef* in 14 of 16 runs.

To diagnose this, we build PercEF, a nonparametric variant that keeps Ada-ef's design but replaces its Gaussian thresholds with fixed ones read from the database's empirical distance percentiles. PercEF ranks queries better on all eight non-Gaussian datasets and never costs more than 2.5% over the tuned fixed *ef*. On near-Gaussian text embeddings, however, Ada-ef buys more tail recall. Because our diagnostic test needs only the raw vectors and runs in seconds, practitioners can run it before building an index. Finally, neither score reliably adapts to cross-modal queries, and we identify several common pitfalls that inflate the measured savings of adaptive search.

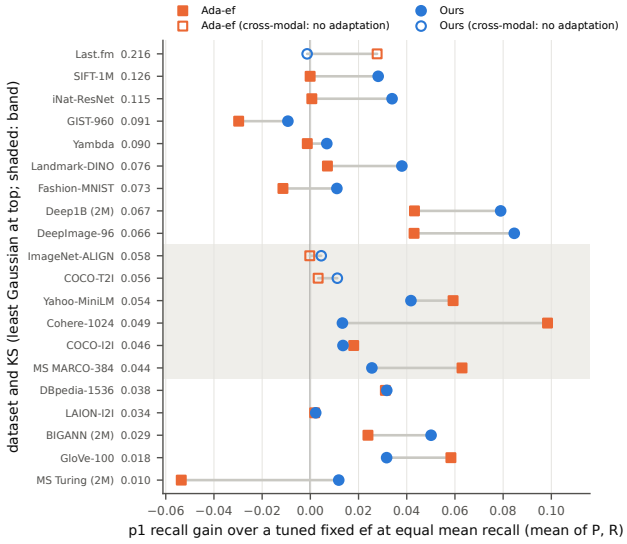


Figure 1: Tail recall gained over a fixed *ef* at the same mean recall, for Ada-ef and for our empirical score, on 20 datasets ordered by how far they are from Gaussian (KS, higher is less Gaussian). Above the shaded band, where Ada-ef's Gaussian model does not hold, the empirical score wins on every dataset whose queries come from the same modality as the database. Below it, on near-Gaussian text embeddings, Ada-ef usually wins. Hollow markers: cross-modal queries, where neither score adapts.

1 Introduction

Graph indexes such as HNSW [14] answer a nearest-neighbor query by a best-first walk whose breadth is set by one parameter, *ef*. Most deployments pick one *ef* for all queries. That choice is wasteful in both directions: an *ef* large enough for the hardest queries spends work on the easy majority, and an *ef* tuned for the average leaves the hardest queries short of the recall the application asked for. Several recent systems therefore choose the search effort per query, either by learning when to stop [5, 12] or by estimating, before the search, how hard the query will be [20].

Ada-ef [20] takes the second route without training a model. It models the similarities between a query and the database as a Gaussian whose mean and variance follow from the database's mean and covariance; a short probe of the graph counts how many of the query's nearby points fall in the extreme tail of that Gaussian, and the count becomes a difficulty score that maps to an *ef*. The method is simple and cheap, and on the OpenAI and Cohere embeddings of its evaluation it reaches its target recall with up to 4× lower latency than learned approaches. This paper asks: *does the Gaussian model behind Ada-ef hold on the data it serves, and what happens to adaptive search when it does not?*

At first sight, the central limit theorem seems to settle the question: a similarity is a sum over hundreds of dimensions. However, the theorem holds only when no few dimensions dominate that sum, and whether real embeddings meet this condition is an empirical question. The Ada-ef paper supports its assumption on the individual dimensions of GloVe and MS MARCO; it does not report how closely the distribution of each query's similarities fits, dataset by dataset, and as far as we can tell no other paper on nearest-neighbor search reports this either: query-hardness measures in the literature [3, 9, 19] either assume such a model or avoid it. We therefore study four questions:

- **Q1.** How often does the Gaussian hold, and does the data type predict it? (§4.2)
- **Q2.** What does Ada-ef lose where it departs? (§4.3)
- **Q3.** Does a nonparametric score recover that loss, and what does it give up where the Gaussian holds? (§4.3, §4.4)
- **Q4.** Which component carries Ada-ef's benefit: its thresholds, its probe or its *ef* table? (§4.5)

We measure the fit with a Kolmogorov–Smirnov (KS) statistic between each query's similarities and the Gaussian Ada-ef predicts,

on 41 datasets from ann-benchmarks, Big-ANN, VIBE and the Ada-ef paper's own sources (Fig. 2). The Gaussian fits most modern text embeddings but departs on nine datasets, six of them learned embeddings (CNN features in DeepImage and Deep1B, DINO and ResNet image models, an audio model in Yambda, a factorisation model in Last.fm). On the eight whose queries share the data's modality, Ada-ef's score loses much of its ability to rank queries by difficulty, and on SIFT and Yambda it gives every query the same *ef*. Against a hindsight-tuned fixed *ef* at the same mean recall, its released implementation does more work in 14 of these 16 runs and lowers the recall of the worst 1% of queries in 10. To test whether the thresholds are responsible, we change only them: PercEF reads them from the database's empirical distance percentiles and keeps the rest of Ada-ef. It ranks queries better on all eight datasets, keeps the worst case at or above the fixed *ef*'s in 13 of the 16 runs (Ada-ef: 6) and, across 17 datasets, never costs more than 2.5% extra (Fig. 1). Where the Gaussian holds, the picture reverses: Ada-ef's score and *ef* table together buy larger tail gains, in 11 of 18 runs. Learned early termination (DARTH, LAET) buys the largest tail gains of all, at up to 2.8 times the time of a fixed *ef*.

The overarching lesson is that *whether the Gaussian holds is a property of the dataset, not of the data type, and a seconds-long test on the raw vectors reveals it before an index is built.* SIFT-1M is the least Gaussian image dataset we measured while SIFT-1B is close to Gaussian, and MNIST fits much better than Fashion-MNIST. Written down before each run, the test's predictions of the better score were right on 18 of 19 datasets.

Contributions.

(1) A per-query test of Ada-ef's Gaussian assumption on 41 public datasets, showing where it holds and that the data type does not predict it (§4.2).
(2) An end-to-end re-evaluation of Ada-ef against a hindsight-tuned fixed *ef*, matched on both mean and worst-case recall, on 17 datasets in two calibration settings, with DARTH and LAET on seven and robustness to other target recalls and k (§4.3–§4.5).
(3) PercEF, a nonparametric variant used as a controlled intervention, and an attribution of Ada-ef's benefit to its thresholds, probe and *ef* table (§4.5).
(4) The KS check, validated by predictions written in advance, together with the negative results (near-Gaussian tails, $k = 10$, cross-modal queries, the SIFT-1B miss) and six pitfalls that inflate measured savings of adaptive search (§4.5, §4.6).

2 Related Work

Adaptive search in graph indexes. LAET [12] trains a gradient-boosted model on features gathered early in the search to predict how much longer each query should run. DARTH [5] also terminates early, but targets a declared recall and checks its prediction periodically during the search. Both need a training phase with ground truth and adapt the stopping point while the search runs. Ada-ef [20] instead decides *ef* once, after a short probe, from a statistical model of the query's similarity distribution; it needs only the database mean and covariance and a small calibration set, which is what makes it cheap and easy to update. PercEF keeps this design. What we examine is the statistical model, whose fit these papers do not report dataset by dataset.

Experimental studies of ANN search. Experimental surveys compare graph indexes [18] and ANN methods across many datasets [2, 13] and show that rankings depend on the data. We follow their approach for one design decision, the per-query *ef*, and add what they do not measure: the distributional assumption a method relies on. Distance distributions and their histograms have long been used to characterise how hard a metric space is to search [6]; AD-Sampling [8] uses the concentration of projected distances to cut distance computations. Queries from another distribution than the data are known to hurt graph indexes [7, 11]; our cross-modal results show they also defeat both difficulty scores.

Query and dataset hardness. He et al. [9] relate the difficulty of nearest-neighbor search to the relative contrast between nearest and mean distances, derived under a Gaussian-type model of the data. Aumüller and Ceccarello [3] show that local intrinsic dimensionality and related measures vary widely across queries in standard benchmarks and change measured performance. Hubness [16] describes how a few points become nearest neighbors of many queries in high dimensions. Steiner-hardness [19] measures the effort a graph index needs for a query from the index itself. These measures describe hardness after the fact or need the index; none asks whether the distributional model behind a cheap online score holds, which is the question we answer with a test that runs on the raw vectors.

3 Methodology

3.1 Preliminaries

Let $X \subset \mathbb{R}^d$ be the database, with vectors normalised to unit length so that cosine similarity and Euclidean distance give the same ranking. For a query q and an integer k, HNSW returns k candidates found by a best-first search whose candidate list holds at most *ef* entries; recall@k is the share of the true k nearest neighbors among them. We write $\mathrm{ef}^*(q)$ for the smallest *ef* on a grid at which q reaches the target recall τ.

Ada-ef's score. Ada-ef models the distance $\delta(q,x) = 1 - q^\top x$ between q and a random database vector x as

$$\delta(q,x) \sim \mathcal{N}\big(1 - q^\top \mu,\ q^\top \Sigma q\big), \tag{1}$$

where μ and Σ are the mean and covariance of X. It places $B = 5$ thresholds $t_1 < \cdots < t_B$ at the $\beta, 2\beta, \ldots, B\beta$ quantiles of this Normal, with $\beta = 10^{-3}$, so that they mark the closest tenth of a percent of the database, the next tenth, and so on. An unpruned best-first probe collects the first $L = 1025$ distances the search meets, $P(q)$, and the score is

$$s(q) = \frac{1}{|P(q)|} \sum_{p \in P(q)} w_{b(p)}, \qquad w_b = 100\, e^{-(b-1)}, \tag{2}$$

where $b(p)$ is the first bin whose threshold p falls below (points beyond t_B add nothing). A query surrounded by many unusually close points scores high and is easy; a query whose probe finds few is hard. Offline, Ada-ef scores a calibration set, groups it by rounded score, and stores for each group the smallest *ef* at which the group's average recall reaches τ. Its paper also floors every

entry at the weighted average of these values (WAE); the released code does not. We report both variants.

3.2 Testing the Gaussian assumption

Equation (1) can be checked directly. For a query q we compute the similarities $q^\top x$ to a uniform sample of the database and the KS statistic between their empirical distribution and the Normal implied by (1),

$$\mathrm{KS}(q) = \sup_z \left| \hat{F}_q(z) - \Phi\left(\frac{z - q^\top \mu}{\sqrt{q^\top \Sigma q}} \right) \right|. \tag{3}$$

We report the mean over 200 test queries, with μ and Σ estimated from up to 10^6 database vectors and $\mathrm{KS}(q)$ from 20,000. With 30 queries the estimate was noisy by as much as the effects we were trying to see, which is why we use 200 and report a 95% interval. The test needs only the vectors and a few seconds per dataset.

3.3 PercEF: a nonparametric control

The thresholds in (2) are the only place where the Gaussian enters Ada-ef's score. To test whether they are what fails, we take them from the data instead and change nothing else. Let $c_1, \ldots, c_C$ be centroids of X ($C = 1$ gives the database mean) and, for each centroid, let $t_b^{(c)}$ be the empirical $b\beta$ quantile of the squared distances $\|x - c\|^2$ of the database vectors assigned to it. At query time we find the nearest centroid, probe the first $L = 100$ distances, and evaluate (2) with these thresholds and the same weights. The thresholds thus describe what "unusually close" means in this particular database, whatever the shape of its distance distribution.

Unlike Ada-ef's, these thresholds do not depend on the query: they turn Ada-ef's per-query quantiles into fixed radii measured on the database, and the score counts how many of the probe's points fall within them. Nothing has to be computed per query beyond the probe itself, where Ada-ef needs $q^\top \Sigma q$, a d^2 computation. A per-query empirical version, which would estimate each query's own quantiles from a sample, is a natural alternative that we leave to future work.

For calibration we measure $\mathrm{ef}^*(q)$ for every calibration query and fit an isotonic regression [4] of ef^* on the rounded score, letting the fit choose the direction, and clip the result to $[k, 5000]$. Optionally, the table is then multiplied by one factor so that the mean recall on the calibration queries meets the target (§4.5). The search does not restart after the probe: it continues the same traversal with the chosen ef, so the probe's distance computations are part of the search, not an addition to it. Our default, fixed before the end-to-end runs, is $C = 1$ with isotonic calibration; we also ran $C \in \{8, 50\}$ and three bucket recipes and treat them as secondary.

The two scores differ in their thresholds, in the probe length (100 against 1025) and in how the ef table is built. The rank correlation we report between score and ef^* depends only on the first two, and we separate them by running each score with the other's probe length (§4.5).

3.4 Implementation

Both methods run inside the same HNSWlib build. Ada-ef runs through its authors' own code: we call their scoring, sketch and search routines unchanged and only add a loader that reads the database statistics from a file, so that they can be computed in one streaming pass. On every dataset that fits in memory we also built Ada-ef's estimator the original way and checked that the two give the same scores (50 of 50 queries identical on all of them). Our score is computed inside the same traversal, with the same memory prefetching as HNSWlib's own search loop. All searches run on one thread.

4 Experiments

4.1 Setup

Datasets. We benchmark 20 datasets end to end (Table 1): the four in the Ada-ef paper that we could obtain (GloVe-100, DeepImage-96, Cohere-1024 and LAION-I2I, the last two as subsets that fit in 62 GB of memory), a MiniLM-384 stand-in for its MS MARCO set, standard ann-benchmarks sets [2], 2M-vector prefixes of three Big-ANN sets [17] (the benchmark builds its own subsets the same way), four VIBE sets [10], and Yambda audio embeddings [15]. Fourteen of them were added after the first six, each with its outcome predicted in advance from KS (§4.5). Three have queries from a different modality than the database (text against images, users against songs).

Protocol. We follow the Ada-ef paper: HNSW with $M = 16$ and $ef_{\text{construction}} = 500$, $k = 1000$ for the MS MARCO, Cohere and LAION sets and $k = 100$ otherwise, target recall $\tau = 0.95$, and ef capped at 5000. Both methods always calibrate on the same set, in two settings: P, 200 database vectors as in the Ada-ef paper, and R, up to 2000 held-out real queries. Test queries are disjoint from both.

Metrics and the fixed-ef reference. Ada-ef's stated aims are to meet a declared target recall while avoiding both under- and over-searching [20]. We use the same recall statistics its evaluation reports: mean recall, and the 1st and 5th percentile of per-query recall (p1, p5), which "capture performance on the hardest queries" and so show under-searching [20, §7.2]. Cost is measured as latency per query, as there, and also as distance computations, which the Ada-ef paper does not report; plus offline time and memory. Distance computations (DC), including every probe, serve as a hardware-independent measure of cost. We count them as HNSWlib does, as every neighbor-list entry a search examines, visited or not; all methods run in HNSWlib are counted the same way. As a reference we run HNSW at a grid of fixed ef values and, for each method, interpolate the fixed ef that reaches the same mean recall. This is the best fixed ef anyone could choose in hindsight; no deployment can pick it in advance, so beating it is a demanding test. The Ada-ef paper instead sweeps the fixed ef upward until mean recall reaches 0.99 and compares the curves, so it does not match the two at equal mean recall. We interpolate in $\log(1 - \text{recall})$ against log cost: interpolating linearly overstates the fixed ef's cost by 11% on average in a leave-one-out test on our grids (§4.6), which would flatter every adaptive method.

4.2 How often does the assumption hold?

Figure 2 shows KS for all 41 datasets. Most lie below 0.044, the level up to which Ada-ef's score ranked queries better than ours in our runs. They are mostly text and text-image embeddings (OpenAI,

Table 1: Datasets, sorted by KS (200 queries). ρ: rank correlation between each score and the true per-query ef* on the calibration set, settings P / R; the larger magnitude ranks queries better. A dash: the score is constant on the calibration queries.

dataset	source	in Ada-ef paper	corpus	dim	K	test queries	KS	rho Ada-ef P	rho ours P	rho Ada-ef R	rho ours R
MS Turing (2M)	Big-ANN	no	2,000,000	100	100	10000	0.010	−0.36	−0.35	−0.48	−0.40
GloVe-100	ann-benchmarks	yes	1,183,514	100	100	8000	0.018	−0.81	−0.61	−0.80	−0.61
BIGANN (2M)	Big-ANN	no	2,000,000	128	100	8000	0.029	−0.42	−0.74	−0.42	−0.74
LAION-I2I	LAION, shards 0-19	yes (subset)	19,639,187	512	1000	10000	0.034	−0.59	−0.56	−0.57	−0.53
DBpedia-1536	DBpedia, OpenAI embeddings	no	990,000	1536	100	8000	0.038	−0.48	−0.38	−0.48	−0.54
MS MARCO-384	MS MARCO (MiniLM)	stand-in	8,841,823	384	1000	6980	0.044	−0.66	−0.53	−0.69	−0.53
COCO-I2I	ann-benchmarks	no	113,287	512	100	8000	0.046	−0.33	−0.39	−0.35	−0.36
Cohere-1024	Cohere, files 00-04	yes (subset)	9,511,868	1024	1000	1174	0.049	−0.59	−0.40	−0.79	−0.52
Yahoo-MiniLM	VIBE	no	677,305	384	100	700	0.054	−0.55	−0.54	−0.50	−0.68
COCO-T2I	ann-benchmarks	no	113,287	512	100	8000	0.056	−0.34	−0.38	–	–
ImageNet-ALIGN	VIBE	no	1,281,167	640	100	1000	0.058	−0.17	−0.23	–	–
DeepImage-96	ann-benchmarks	yes	9,990,000	96	100	8000	0.066	−0.42	−0.63	−0.38	−0.62
Deep1B (2M)	Big-ANN	no	2,000,000	96	100	8000	0.067	−0.39	−0.64	−0.41	−0.67
Fashion-MNIST	ann-benchmarks	no	60,000	784	100	8000	0.073	−0.20	−0.22	−0.17	−0.29
Landmark-DINO	VIBE	no	760,757	768	100	700	0.076	−0.45	−0.61	−0.34	−0.55
Yambda	Yambda	no	7,709,749	128	100	10000	0.090	+0.02	−0.41	−0.03	−0.43
GIST-960	ann-benchmarks	no	1,000,000	960	100	700	0.091	−0.10	−0.46	+0.08	−0.23
iNat-ResNet	VIBE	no	499,000	2048	100	700	0.115	−0.24	−0.46	−0.21	−0.46
SIFT-1M	ann-benchmarks	no	1,000,000	128	100	8000	0.126	+0.08	−0.33	−0.04	−0.45
Last.fm	ann-benchmarks	no	292,385	65	100	10000	0.216	+0.07	+0.23	–	–

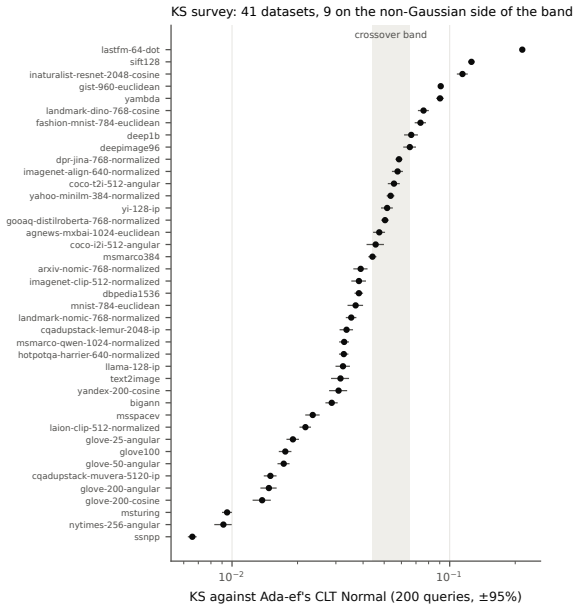


Figure 2: KS between each query's similarities and Ada-ef's Gaussian, mean over 200 queries with 95% interval, for 41 public datasets. Shaded: the band in which neither score ranks queries clearly better. GloVe-200 appears twice, from ann-benchmarks and from VIBE.

Cohere, MiniLM, Nomic, CLIP and others), but word vectors, LLM attention keys and SIFT-1B are among them too. Nine lie at or above the band's upper edge, set by DeepImage at 0.066: SIFT-1M, GIST, Fashion-MNIST, DeepImage, Deep1B, iNaturalist ResNet, Landmark DINO, Yambda and Last.fm. Between the two is a band where neither wins clearly. Datasets built from the same kind of data land on different sides (SIFT-1M at 0.126, SIFT-1B at 0.029; Fashion-MNIST at 0.073, MNIST at 0.037), so the assumption has to be checked on the data itself.

4.3 Comparison on non-Gaussian datasets

Table 2 summarises every run against the fixed-*ef* reference, and Fig. 3 shows where each method sits relative to the fixed-*ef* curve.

Table 2: Runs in which each method is cheaper than the fixed *ef* at the same mean recall (DC), faster (latency, every run timed), and has at least its p1 tail recall; and runs in which it is faster than the fixed *ef* with the same p1 (p1-matched). Same modality unless marked. Worst: the largest extra cost.

		same mean recall				p1-matched
Datasets	Method	Cheaper	Worst	Faster	p1 ≥	Faster
All (34 runs)	Ours	32/34	−2.5%	26/34	31/34	31/34
	Ada-ef	10/34	−24.6%	3/34	22/34	11/34
	Ada-ef WAE	17/34	−24.6%	8/34	23/34	14/34
KS ≥ 0.066 (16)	Ours	14/16	−2.5%	8/16	13/16	13/16
	Ada-ef	2/16	−24.6%	0/16	6/16	1/16
Band (8)	Ours	8/8	+3.4%	8/8	8/8	8/8
	Ada-ef	4/8	−7.4%	2/8	8/8	6/8
KS < 0.044 (10)	Ours	10/10	+1.9%	10/10	10/10	10/10
	Ada-ef	4/10	−12.4%	1/10	8/10	4/10
	Ada-ef WAE	8/10	−2.1%	5/10	8/10	6/10
Cross-modal (6)	Ours	1/6	−1.0%	0/6	5/6	4/6
	Ada-ef	0/6	−10.3%	0/6	4/6	0/6

On the eight datasets above the band (16 runs), our score has the larger rank correlation with ef* in all 16. Its p1 is at or above the fixed *ef*'s in 13 of the 16 runs, against 6 for Ada-ef; in the other three (GIST P and R, Yambda P) it is lower by at most 0.015, where Ada-ef's p1 is lower than the fixed *ef*'s in 10 runs, by up to 0.046. Compared directly, PercEF's change in p1 is at least Ada-ef's in 15 runs; on GIST that means a smaller loss, not a gain. The gains are often large: p1 rises by 0.085 over the fixed *ef* on DeepImage (Ada-ef 0.043), by 0.079 on Deep1B (0.043) and by 0.034 on iNaturalist (0.001). Ada-ef costs more than the fixed *ef* in 14 of these 16 runs; ours costs less in 14, and the two exceptions are small (DeepImage P, 2.5% more; GIST R, 0.8% more).

Matching the fixed *ef* on mean recall credits PercEF with the work it saves but not with the tail it buys: on DeepImage it spends most of its savings raising p1 by 0.085. Matched instead on p1, that is, against the cheapest fixed *ef* that serves the worst 1% of queries as well, PercEF needs less work and less time in 13 of the 16 runs, a median of 14% less time (24–31% on DeepImage and Deep1B). The exceptions are GIST and Yambda P, where PercEF does not raise p1. Ada-ef is faster than the fixed *ef* matched on its own p1 in one of the 16 runs. We report both comparisons throughout.

On SIFT-1M and Yambda, Ada-ef's score collapses: every query lands in the same group and gets the same *ef*. Ada-ef then behaves as a fixed *ef* that also pays for a 1025-distance probe. Ours still separates queries, with rank correlations of 0.33 to 0.45 against ef*. The gains are not uniform. On GIST both scores do little for the tail (ours −0.009, Ada-ef −0.030), and on Fashion-MNIST most queries reach the target at the smallest *ef*, which leaves little room for any adaptive method.

4.4 Comparison on near-Gaussian datasets

Below the band the comparison becomes a trade-off. PercEF is cheaper and faster than the fixed *ef* in all 18 runs in or below the band, and never lowers its tail; Ada-ef as shipped is cheaper in 8 of them and faster in 3. But Ada-ef's tail gains are larger in 11 of the 18, most clearly on GloVe, MS MARCO and Cohere: p1 gains of 0.058, 0.063 and 0.098 over the fixed *ef*, against our 0.032, 0.026 and

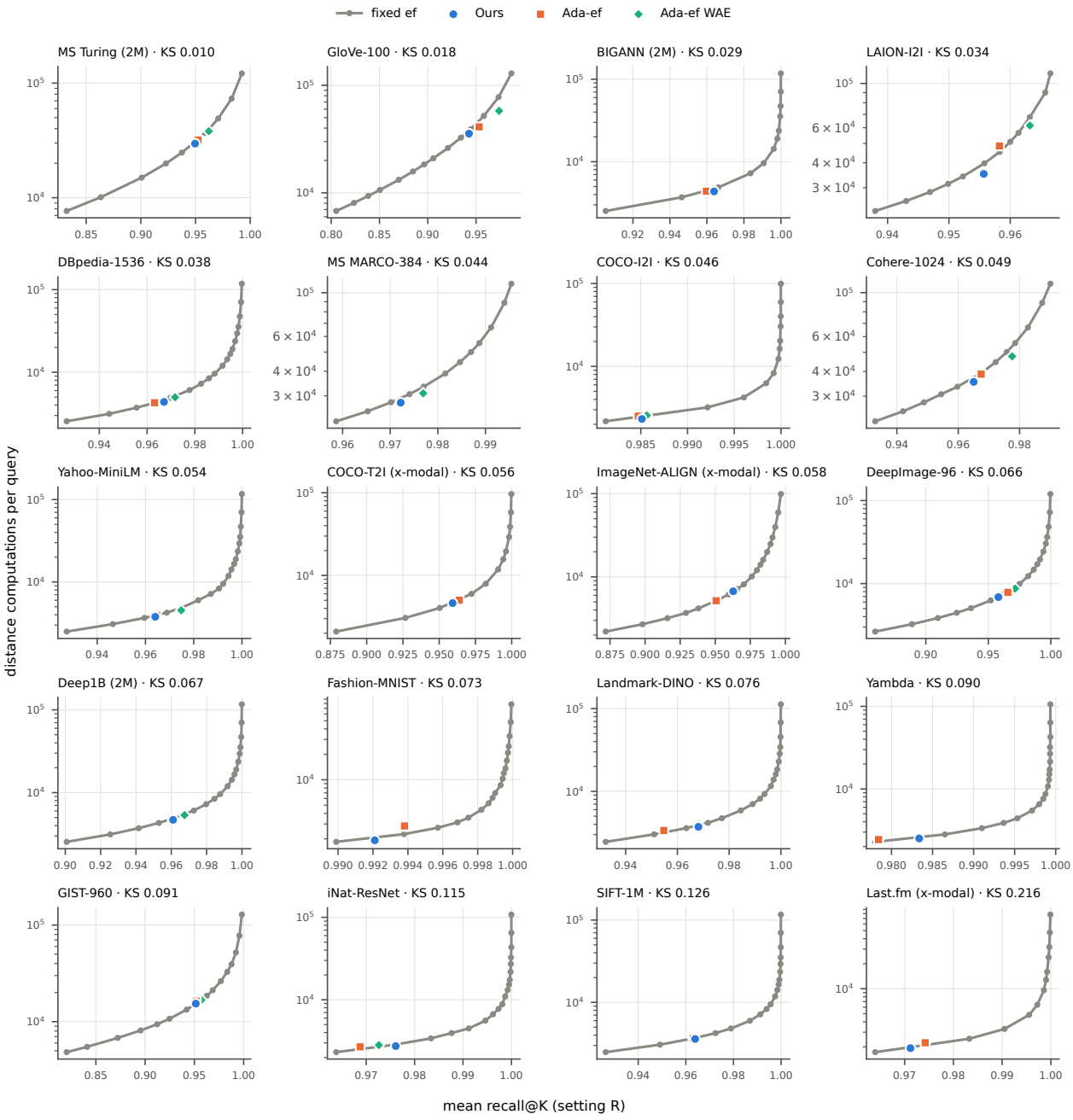


Figure 3: Distance computations per query against mean recall, setting R. Grey: HNSW with fixed *ef*. Points below the curve reach the same recall with less work. Panels are ordered by KS, from near-Gaussian (top left) to non-Gaussian (bottom right).

0.013. With its WAE floor Ada-ef is also cheaper than the fixed *ef* by a wide margin on these sets (27% on GloVe, 7% on MS MARCO, 3–11% on Cohere and LAION), and faster in wall-clock time (§4.5). Matched on p1, Ada-ef's larger tail gain pays off inside the band, where it is faster than the p1-matched fixed *ef* in 6 of 8 runs, but less below it (4 of 10); PercEF is faster than its own p1-matched fixed *ef* in all 18, by a median of 15%. We therefore do not claim that the empirical score should replace Ada-ef's everywhere: on text embeddings like those in the Ada-ef paper, Ada-ef is the better choice when the worst-served queries matter most, and PercEF when a predictable cost does. The ablation below qualifies why: part of Ada-ef's ranking edge there comes from its longer probe, and its tail edge needs its own score and its *ef* table together.

Two datasets on this side behave differently. On SIFT-1B (KS 0.029) our score ranks queries far better (0.74 against 0.42) and has the better tail. On MS Turing (KS 0.010) Ada-ef ranks better, as expected, yet its tail is worse than the fixed *ef*'s (p1 −0.067 and −0.041) because its group-average table under-provisions some score groups; with the WAE floor it is level.

4.5 Analysis

Predicting the better score. Across datasets, KS correlates with Ada-ef's rank correlation at −0.87 (13 datasets, setting R). To use it as a rule, we set the band from the first six datasets and wrote down a prediction for each of the other fourteen before running it. It held for Landmark-DINO, iNaturalist, GIST, Fashion-MNIST and Deep1B on our side, and for LAION, Cohere and MS Turing on Ada-ef's; Yahoo-MiniLM and COCO-I2I fell in the band as predicted and split or tied; three cross-modal sets could not be scored (below); and SIFT-1B failed. Over the 19 datasets with a clear winner, a single KS threshold sorts 18 correctly, and 16 when each threshold is fitted without the dataset it predicts.

We expected the SIFT-1B failure to come from the tail: Ada-ef's thresholds sit at the 0.1% quantiles, where KS, dominated by the bulk of the distribution, says little. So we measured the tail directly, with Anderson–Darling [1], the error of the Normal's upper quantiles, and the mass beyond its 99.9% point (Table A2 in the supplementary appendix). None did better than always predicting our score, and the reason is in the numbers: the text embeddings on which Ada-ef wins have the least Gaussian upper tails (MS MARCO +2.3 standard deviations at the 10^{-4} quantile), while SIFT-1B's tail is close to Gaussian. How well the Normal fits the tail is not what decides whether the score works, and we do not yet know what makes SIFT-1B different. A cheap check covers the case in practice: Ada-ef's calibration already scores a few hundred points, and when those scores take only a handful of distinct values (5 to 11 on SIFT-1B, against 60 to 100 on datasets where the score works), the score cannot rank queries.

Which component carries the benefit? Ada-ef's score has three parts: Gaussian thresholds, a 1025-distance probe and a group-average *ef* table. PercEF changes the first and also probes 100 distances, so its better ranking could come from the probe rather than from the thresholds. We therefore ran each score with the other's probe length (Table 3). On SIFT-1M, Deep1B, DeepImage and SIFT-1B the empirical thresholds rank queries better at both lengths, by 0.21 to 0.47: the thresholds, not the probe, carry the advantage. On the near-Gaussian sets the probe matters more than we assumed. A longer probe helps both scores there, and at equal length the empirical thresholds tie the Gaussian ones on GloVe at 100 distances and beat them on MS Turing at both lengths; only on GloVe at 1025 do the Gaussian thresholds rank better (0.80 against 0.76). Better ranking did not carry over to the tail, though: with 1025 distances PercEF's p1 gain on GloVe stays at 0.028, against Ada-ef's 0.073. Nor is it Ada-ef's table alone: with the table Ada-ef builds (each score group gets the smallest *ef* at which the group's average recall reaches the target), PercEF's p1 gain on GloVe is 0.031. Ada-ef's tail on these datasets needs its score and its table together. On the non-Gaussian sets the same table lowers PercEF's tail slightly (e.g. DeepImage 0.088 to 0.073), so we keep PercEF's own calibration. A long probe is cheap only when searches are long: with 1025 distances PercEF is 26–40% slower than the fixed *ef* on SIFT-1M, DeepImage, SIFT-1B and Deep1B, and about 1% slower on GloVe and MS Turing.

Table 3: Rank correlation $|\rho|$ between each score and ef* when both scores use each probe length L (setting R).

		Ada-ef		PercEF	
Dataset	KS	L=1025	L=100	L=100	L=1025
SIFT-1M	0.126	0.04	0.03	**0.45**	**0.40**
Deep1B	0.067	0.41	0.43	**0.67**	**0.63**
DeepImage	0.066	0.38	0.33	**0.62**	**0.59**
SIFT-1B	0.029	0.42	0.27	**0.74**	**0.72**
GloVe	0.018	**0.80**	0.62	0.61	0.76
MS Turing	0.010	0.48	0.21	**0.40**	**0.58**

Latency and offline cost. Ada-ef's wall-clock cost depends on how long the searches are. Its probe is a fixed 1025 distances; on searches of several milliseconds that is a small share, and there the WAE variant is faster than the fixed *ef* at the same recall: by 21% on GloVe and by up to 4–5% on MS MARCO, Cohere and LAION, all datasets of the Ada-ef paper or stand-ins for them. On short searches the probe dominates: Ada-ef is 8–103% slower than the fixed *ef* on the non-Gaussian datasets and 87% slower on Last.fm, where a whole search takes 0.13 ms. At the same mean recall, PercEF is faster than the fixed *ef* in 26 of 34 runs: all 18 on datasets in or below the band (by 0.3% to 6%) and 8 of the 16 non-Gaussian runs. In the other 8 it is slower by at most 2.4%, except on DeepImage (7.1 and 8.9%). Above the band these counts are fragile: timing the same searches twice moved PercEF's saving by up to 3.4 points and Ada-ef's by up to 16, so 8 of 16 is not a stable figure.

Why does the time PercEF saves lag the work it saves? Not every distance computation costs the same. Under a plain fixed *ef*, the queries PercEF classifies as hard (top 20% of its *ef*) take a median 8% more time than fixed-*ef* queries with the same distance count, and the queries it calls easy 11% less (Table A1), probably because hard queries wander through less cached parts of the graph. PercEF moves work from easy queries to hard ones, so it saves cheap distance computations and spends dearer ones: its time per distance computation is a median 2% above the fixed *ef*'s, about the same above and below the band. Below the band it saves enough work at equal mean recall (median 5%) to cover this; above it, where its savings go into the tail, it saves less (median 2.5%). Matched on p1 the saving is larger, and so is the time saved (Table 2). The only per-query overhead we found in the code, a copy of the result heap used for scoring, is now removed; results were identical and the time did not change measurably. Against Ada-ef directly, PercEF is faster in 31 of 34 runs, by 0.99 to 1.82 times in the 18 runs where both reach nearly the same mean recall. Offline, both methods need seconds for their statistics. Calibration time is dominated by measuring ef* (ours, up to 257 s on MS Turing) and by Ada-ef's per-group search (206 s on the same set). Ada-ef keeps a $d \times d$ covariance (up to 9.4 MB at $d = 1536$); ours keeps five thresholds, the centroid and the table, a few kilobytes.

Other targets and k. All results so far use the Ada-ef paper's protocol ($\tau = 0.95$, $k = 100$ or 1000). We repeated the comparison at

$\tau = 0.90$ and 0.99 and at $k = 10$ on six datasets, four above the band (DeepImage, Deep1B, SIFT-1M, Landmark-DINO) and two below it (GloVe, MS Turing), in both settings (Table 4). At both other targets the picture holds. Above the band PercEF does less work than the tuned fixed *ef* in 7 and 6 of the 8 runs (never more than 2.2% extra), never lowers its p1, and is faster than the p1-matched fixed *ef* in all 8, by a median of 19–20%; Ada-ef does less work in 0 and 2. Below the band PercEF wins on every count in all 4 runs at both targets.

Two weaknesses appear. First, PercEF's mean recall lands further from the target than Ada-ef's: at $\tau = 0.90$ it reaches 0.93–0.96 above the band (Ada-ef 0.89–0.95), and at $\tau = 0.99$ it falls short on GloVe (0.967–0.970) and MS Turing P (0.964), where Ada-ef reaches 0.972–0.982. The reason is in the calibration: our table maps a score to the average ef* of similar calibration queries and never checks the mean recall that results, while Ada-ef's table is built to meet the target. Every comparison above is made at the method's own recall, so the counts are not affected. The gap closes with one offline step: we multiply the whole table by a single factor, the smallest whose mean recall on the calibration queries reaches τ (found by bisection; nothing changes per query). With it, PercEF's mean recall lands within 0.01 of the target in 7 of the 8 runs above the band at $\tau = 0.95$ (Ada-ef 3) and in all 8 at $\tau = 0.99$ (Ada-ef 8), and in 3 of the 4 below the band at each target (Ada-ef 2). Its advantage over the fixed *ef* survives: above the band it still does less work in 6 and 7 of the 8 runs (at most 4.5% extra) and is faster than the p1-matched fixed *ef* in all 8, by a median of 17–19%. At $\tau = 0.90$ even $ef = k$ exceeds the target on SIFT-1M, Landmark-DINO and Deep1B, and the scaled table correctly gives every query $ef = k$.

Second, at $k = 10$ neither method saves work: PercEF needs up to 19% more than the tuned fixed *ef* above the band (0 of 8 runs cheaper) and Ada-ef 1–109% more in all 12 runs. The probe is not the cause: with a 30-distance probe PercEF is still cheaper in none of the 8 runs. A likely reason is that with recall moving in steps of 0.1 per query, each query's ef* is too coarse for any score to predict; we leave small k open. The p1-matched comparison is not meaningful at $k = 10$ for the same reason.

Learned methods. DARTH [5] stops each search when a gradient-boosted model, fed with features of the search so far, predicts that the target recall has been reached; LAET [12] predicts once, after a fixed number of distance computations, how many the query needs in total. We ran both through DARTH's authors' FAISS fork, which contains LAET's search, unchanged apart from a build path and a data loader, on seven datasets: GloVe, MS Turing and DBpedia below the band and DeepImage, Deep1B, Landmark-DINO and SIFT-1M above it. They use the same test queries and ground truth, $M = 16$, $ef_{\text{construction}} = 500$ and one thread, are trained on 1,500 calibration queries (DARTH's paper uses 10,000) and tuned on the other 500: DARTH's prediction intervals as in its authors' scripts, LAET's prediction point and multiplier so that validation recall meets the target. Each method is compared with a fixed *ef* of its own library, because the two HNSW implementations build different graphs, time queries differently and count work differently: FAISS counts only the distances it computes, and at the same recall its fixed *ef* reports 0.59 to 0.71 times HNSWlib's count on these datasets (Table 5).

Table 4: Robustness sweep, six datasets, settings P and R. Counts against the tuned fixed *ef*: less work at the same mean recall, p1 at least the fixed *ef*'s, and faster than the p1-matched fixed *ef* (median time saving). Above: 4 datasets above the band (8 runs); below: 2 below it (4).

τ, k	Data	Method	Less work	p1 ≥	p1-matched faster
0.90, 100	above	Ours	7/8	8/8	8/8 (19%)
		Ada-ef	0/8	4/8	0/8
	below	Ours	4/4	4/4	4/4 (11%)
		Ada-ef	0/4	2/4	1/4
0.95, 100	above	Ours	7/8	8/8	8/8 (26%)
		Ada-ef	2/8	5/8	1/8
	below	Ours	4/4	4/4	4/4 (10%)
		Ada-ef	2/4	2/4	1/4
0.99, 100	above	Ours	6/8	8/8	8/8 (20%)
		Ada-ef	2/8	7/8	4/8
	below	Ours	4/4	4/4	4/4 (11%)
		Ada-ef	2/4	2/4	2/4
0.95, 10	above	Ours	0/8	8/8	–
		Ada-ef	0/8	5/8	–
	below	Ours	2/4	3/4	–
		Ada-ef	0/4	2/4	–

Both learned methods buy large tail gains: DARTH has the best p1 gain on six of the seven datasets, LAET on the seventh. They pay for it in time. Both are slower than their fixed *ef* on every dataset but one (LAET on GloVe, 1.3% faster), by up to 2.8 times where a search takes about a millisecond and the model calls dominate, and both do less work than the fixed *ef* only on the two long-search datasets, GloVe and MS Turing (by 7–18%), and 3–20% more on the other five. This agrees with the Ada-ef paper's finding that Ada-ef is much faster than learned methods. Each also needs 2 to 10 minutes of training and tuning per dataset and gigabytes of search traces. PercEF is the only method that does less work than its fixed *ef* on all seven datasets (0.5–5.9%); it is faster on five and at most 7% slower on the others, with no training. Its tail gain is smaller than DARTH's but comes close on Deep1B (0.081 against 0.098).

Cross-modal queries. On Last.fm (user vectors against song vectors), COCO text-to-image and ImageNet ALIGN text-to-image, both scores are constant on real queries: every query gets the same *ef*. Real queries lie far from every database vector, so no probe distance reaches the region either score distinguishes. This is a limitation of both methods. Ours then behaves as a fixed *ef* at about the right value (between 1.3% more and 0.5% less work); Ada-ef also pays for its probe, up to 10% more work and 87% more time.

Controlled data. Varying the distribution of synthetic data moves the ranking advantage in the expected direction in most but not all runs (Fig. A1 in the supplementary appendix); one cluster-separation run at KS 0.10 favours Ada-ef. We treat it as supporting evidence, not as proof.

4.6 Pitfalls in evaluating adaptive search

Several of our own intermediate results did not survive closer measurement, and the reasons apply to any adaptive-*ef* method. Interpolating the fixed-*ef* reference linearly between grid points credited

Table 5: Learned methods (DARTH, LAET; FAISS), Ada-ef (as shipped) and PercEF (HNSWlib) against a tuned fixed *ef* of their own library at the same mean recall (setting R). Positive savings: cheaper or faster than the fixed *ef*. Bold: best per dataset. GloVe, MS Turing, DBpedia: below the band; the rest above it.

Dataset	Method	p1 gain	DC saving	Latency saving
GloVe	DARTH	**+0.080**	+15.5%	−6.3%
	LAET	+0.041	**+17.6%**	+1.3%
	Ada-ef	+0.073	+11.5%	+0.0%
	PercEF	+0.029	+5.9%	**+1.4%**
MS Turing	DARTH	+0.044	+6.7%	−41.9%
	LAET	**+0.051**	**+16.4%**	−16.2%
	Ada-ef	−0.041	+0.1%	−1.2%
	PercEF	+0.010	+1.9%	**+0.9%**
DBpedia	DARTH	**+0.067**	−20.3%	−62.9%
	LAET	+0.057	−8.4%	−60.0%
	Ada-ef	+0.033	−0.5%	−45.5%
	PercEF	+0.034	**+5.3%**	**+2.0%**
DeepImage	DARTH	**+0.119**	−10.4%	−112%
	LAET	+0.097	−5.0%	−115%
	Ada-ef	+0.048	**+1.2%**	−11.5%
	PercEF	+0.088	+0.5%	**−7.1%**
Deep1B	DARTH	**+0.098**	−9.4%	−92.1%
	LAET	+0.090	−10.4%	−55.8%
	Ada-ef	+0.052	+0.3%	−23.1%
	PercEF	+0.081	**+3.7%**	**−2.4%**
Landmark-DINO	DARTH	**+0.078**	−12.0%	−179%
	LAET	+0.034	−11.5%	−157%
	Ada-ef	−0.001	−6.2%	−36.5%
	PercEF	+0.039	**+4.0%**	**+1.8%**
SIFT-1M	DARTH	**+0.048**	−2.8%	−126%
	LAET	+0.031	−8.1%	−78.2%
	Ada-ef	−0.000	−0.2%	−25.2%
	PercEF	+0.028	**+2.7%**	**+0.4%**

every method with 3–4% of savings it did not have; we found it because two methods that assign one *ef* to every query were credited with savings over a fixed *ef*. Timing each method once, in sequence, made latency vary by 24% between identical runs; we time in three rounds with the method order rotated and take per-query medians. KS from 30 queries was noisy by as much as the width of the band. Probe cost must be counted: Ada-ef's fixed probe is a large share of a short search. And about 2% of LAION queries cannot reach the target at any *ef* up to the cap, which limits every method's p1 there; we report LAION's tail at p5. Finally, matching on mean recall alone hides what a method buys in the tail, and matching on p1 alone hides what it costs on average; an adaptive method moves work between queries, so both are needed.

5 Lessons

Test the assumption per dataset, not per data type. The Gaussian holds for most modern text embeddings and fails for several learned image, audio and recommendation embeddings, and closely related datasets fall on opposite sides (SIFT-1M and SIFT-1B, MNIST and Fashion-MNIST). A KS test on a few hundred queries, which needs no index, tells which side a dataset is on.

A cheap score is only as good as its thresholds. Where the thresholds sit in the wrong place, the probe and the table cannot recover: Ada-ef's score is constant on SIFT and Yambda. Taking the thresholds from the data restores the ranking; the longer probe helps only where the model already fits.

Match the baseline on both mean and tail recall. At equal mean recall an adaptive method is credited with the work it saves but not with the tail it buys; at equal tail recall the reverse. Reporting both, against a fixed *ef* tuned with hindsight, separates methods that save work from methods that move it to the hardest queries.

Work saved is not time saved. Probes, model calls and the higher cost per distance of hard queries all sit between the two. Short searches make every fixed overhead visible: learned early termination is up to 2.8 times as slow as a fixed *ef* there, while saving work only on long searches.

When queries come from another modality, nothing adapts. Both scores become constant on cross-modal queries; a well-chosen fixed *ef* is the safe choice there.

6 Conclusion

We asked whether the Gaussian model behind Ada-ef holds on the data it serves. On 41 datasets it fits most modern text embeddings and departs on nine, six of them learned embeddings, and the data type does not tell which. Where it departs, Ada-ef's score loses its ranking ability and its released implementation does more work than a hindsight-tuned fixed *ef* in 14 of 16 runs. Replacing only the thresholds with the data's own percentiles restores the ranking, keeps the worst case at or above the fixed *ef*'s in 13 of 16 runs and never costs more than 2.5% extra. Where the Gaussian holds, Ada-ef's score and table together buy larger tail gains. A seconds-long KS test on the raw vectors tells which case applies before an index is built. The comparisons hold at target recalls of 0.90 and 0.99; neither score saves work at $k = 10$ or adapts to cross-modal queries. Code, data pointers and every number in this paper are generated from the released result files. Available at https://anonymous.4open.science/r/PercEF-SIGMOD27.

References

[1] T. W. Anderson and D. A. Darling. 1952. Asymptotic Theory of Certain "Goodness of Fit" Criteria Based on Stochastic Processes. *The Annals of Mathematical Statistics* 23, 2 (1952), 193–212.

[2] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2020. ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms. *Information Systems* 87 (2020).

[3] Martin Aumüller and Matteo Ceccarello. 2019. The Role of Local Intrinsic Dimensionality in Benchmarking Nearest Neighbor Search. In *Similarity Search and Applications (SISAP)*. 113–127.

[4] R. E. Barlow, D. J. Bartholomew, J. M. Bremner, and H. D. Brunk. 1972. *Statistical Inference under Order Restrictions.* Wiley.

[5] Manos Chatzakis, Yannis Papakonstantinou, and Themis Palpanas. 2026. DARTH: Declarative Recall Through Early Termination for Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data (SIGMOD)* (2026). arXiv:2505.19001.

[6] Edgar Chávez, Gonzalo Navarro, Ricardo Baeza-Yates, and José Luis Marroquín. 2001. Searching in Metric Spaces. *ACM Comput. Surv.* 33, 3 (2001), 273–321. doi:10.1145/502807.502808

[7] Meng Chen, Kai Zhang, Zhenying He, Yinan Jing, and X. Sean Wang. 2024. RoarGraph: A Projected Bipartite Graph for Efficient Cross-Modal Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 17, 11 (2024), 2735–2749. doi:10.14778/3681954.3681959

[8] Jianyang Gao and Cheng Long. 2023. High-Dimensional Approximate Nearest Neighbor Search: with Reliable and Efficient Distance Comparison Operations. *Proc. ACM Manag. Data* 1, 2 (2023), 1–27. doi:10.1145/3589282

[9] Junfeng He, Sanjiv Kumar, and Shih-Fu Chang. 2012. On the Difficulty of Nearest Neighbor Search. In *Proceedings of the 29th International Conference on Machine Learning (ICML).*

[10] Elias Jääsaari, Ville Hyvönen, Matteo Ceccarello, Teemu Roos, and Martin Aumüller. 2025. VIBE: Vector Index Benchmark for Embeddings. arXiv:2505.17810.

[11] Shikhar Jaiswal, Ravishankar Krishnaswamy, Ankit Garg, Harsha Vardhan Simhadri, and Sheshansh Agrawal. 2022. OOD-DiskANN: Efficient and Scalable Graph ANNS for Out-of-Distribution Queries. arXiv:2211.12850

[12] Conglong Li, Minjia Zhang, David G. Andersen, and Yuxiong He. 2020. Improving Approximate Nearest Neighbor Search through Learned Adaptive Early Termination. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 2539–2554.

[13] Wen Li, Ying Zhang, Yifang Sun, Wei Wang, Mingjie Li, Wenjie Zhang, and Xuemin Lin. 2020. Approximate Nearest Neighbor Search on High Dimensional Data — Experiments, Analyses, and Improvement. *IEEE Trans. Knowl. Data Eng.* 32, 8 (2020), 1475–1488. doi:10.1109/TKDE.2019.2909204

[14] Yu. A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836.

[15] A. Ploshkin, V. Tytskiy, A. Pismenny, V. Baikalov, E. Taychinov, A. Permiakov, D. Burlakov, E. Krofto, and N. Savushkin. 2025. Yambda-5B – A Large-Scale Multi-modal Dataset for Ranking and Retrieval. arXiv:2505.22238.

[16] Miloš Radovanović, Alexandros Nanopoulos, and Mirjana Ivanović. 2010. Hubs in Space: Popular Nearest Neighbors in High-Dimensional Data. *Journal of Machine Learning Research* 11 (2010), 2487–2531.

[17] Harsha Vardhan Simhadri et al. 2022. Results of the NeurIPS'21 Challenge on Billion-Scale Approximate Nearest Neighbor Search. In *Proceedings of the NeurIPS 2021 Competitions and Demonstrations Track (PMLR, Vol. 176)*. 177–189.

[18] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A Comprehensive Survey and Experimental Comparison of Graph-Based Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 14, 11 (2021), 1964–1978. doi:10.14778/3476249.3476255

[19] Zeyu Wang, Qitong Wang, Xiaoxing Cheng, Peng Wang, Themis Palpanas, and Wei Wang. 2024. Steiner-Hardness: A Query Hardness Measure for Graph-Based ANN Indexes. *Proceedings of the VLDB Endowment* 17, 13 (2024). arXiv:2408.13899.

[20] Chao Zhang and Renée J. Miller. 2026. Distribution-Aware Exploration for Adaptive HNSW Search. *Proceedings of the ACM on Management of Data (SIGMOD)* (2026). doi:10.1145/3786639 Extended version: arXiv:2512.06636.